JAVA BACKEND PORTFÖY PROJESİ

HookRelay

Güvenilir webhook teslimat platformu

Bir olayın yalnızca gönderilmesini değil, teslimat sürecinin izlenmesini ve hatalardan sonra devam etmesini tasarla.

Java + Spring Boot | PostgreSQL | RabbitMQ

ÖĞREN → UYGULA → KANITLA

01
Olayı kabul et
API ve kalıcı kayıt

→

02
Güvenle işle
Outbox, kuyruk, retry

→

03
Sonucu göster
Geçmiş, test, metrik

7 ayda tamamlanabilir bir çekirdek

Haftada 8-10 saat varsayımı. Her ay çalışan bir çıktı; son ayda düzeltme ve sunum için tampon. Bonuslar, çekirdek tamamlandıktan sonra.

HIZLI ERİŞİM

Kapsam 04 | Akış 07 | Sözlük 14 | Yol haritası 16 | GitHub 18

A4 GÖRSEL RAPOR • SÜRÜM 2 • 12 EKİM 2026

StockFlow ile tamamlayıcı yetkinlikler için hazırlanmıştır.

Bir olay kaybolmasın

HookRelay, uygulamaların dış sistemlere gönderdiği HTTP bildirimlerini yönetir.

Gerçek problem

Alıcı sistem kapalı olabilir, ağ zaman aşımına uğrayabilir veya yanıt kaybolabilir. Tek bir HTTP çağrısı, güvenilir bir entegrasyon süreci oluşturmaz.

ÖRNEK SENARYO / SAAS RAPORLAMA UYGULAMASI



01

Kabul

İstek doğrulanır; subscription eşleşmesiyle olay ve teslimat işleri kaydedilir. API hızlıca kabul yanıtı verir.

02

Teslim

Worker kuyruktaki işi alır, imzalı HTTP isteği gönderir ve denemeyi kaydeder.

03

Toparlanma

Geçici hata için bekleyip tekrar dener. Sınır aşılınca nedenini görünür kılar.

Kullanıcıya sunduğun değer

“Hangi olay, hangi hedefe, kaçıncı denemede ulaştı?” sorusunu API, teslimat geçmişi ve metriklerle yanıtlayabilmek.

Neden portföyde güçlü? Java ve REST temellerini; mesajlaşma, kalıcı retry, dağıtık tekrarlar ve hata senaryolarıyla aynı projede uygulamana fırsat verir. İşe alımda anlatılabilir somut kararlar üretir.

StockFlow ile nasıl ayrışır?

Aynı temel becerileri derinleştirirken farklı bir sistem problemini çözersin.

Boyut	StockFlow kapsamında	HookRelay önerisinde
İş alanı	Mini ERP; ürün, sipariş ve stok yönetimi.	Sistemler arası olay ve webhook teslimatı.
Ana etkileşim	ASP.NET Core MVC; Razor formları ve ekranlar.	Spring Boot REST API; producer ve HTTP alıcısı.
Mimari	Controller → Service → EF Core → SQL Server.	Modüler monolit; PostgreSQL, RabbitMQ ve worker.
Kritik doğruluk	Sipariş durumu, stok ve hareketler tek atomik işlemde.	Event + delivery + outbox tek işlemde; sonrasında asenkron teslim.
Tekrarlar	Tekrar onay terminal durumla reddedilir; seed idempotenttir.	Tekrar kabul, kuyruk kopyaları ve belirsiz HTTP sonucu ele alınır.
Güvenlik	Identity, cookie oturumu ve Admin/Employee yetkileri.	Tek organizasyon API key'i, erişim kontrolü, HMAC ve güvenli hedef URL.
Test odağı	Stok yetersizliği, atomiklik, fiyat ve rol kuralları.	Outbox, timeout, retry, duplicate ve worker çökmesi.
İşletim	Logging, migration ve kurulum; Docker/deployment bonus.	Teslimat geçmişi, kuyruk takibi, metrikler ve tekrarlanabilir demo.

İki projeyle anlatacağın yetkinlik

StockFlow: iş kuralları ve ilişkisel bütünlük. HookRelay: API tasarımı, asenkron işleme ve hata karşısında toparlanma. SQL, doğrulama ve test ortak temellerdir.

StockFlow karşılaştırması kapsam belgelerine dayanır; mevcut kodun tamamlanmış olduğunu varsaymaz. Optimistic concurrency, API ve geniş integration testleri StockFlow'da bonus kapsamındadır.

Önce çekirdeği tamamla

Tek organizasyon bağlamı yeterli. Aşağıdaki ayrım tüm API, şema ve plana uygulanır.

01 Güvenli REST API

DTO, doğrulama, API key ve Idempotency-Key; Event/Delivery sorguları.

02 Endpoint + subscription

Bir endpoint, birden fazla event türü. Yalnız etkin ve eşleşen hedefe delivery.

03 PostgreSQL + outbox

Event, Delivery ve Outbox atomik; FK/unique, migration ve attempt geçmişi.

04 RabbitMQ + worker

Confirm/ACK, HTTP teslimatı, claim/lease ve temel lease recovery.

05 Otomatik retry

Kalıcı next_attempt_at, sınırlı deneme, backoff/jitter ve FAILED görünürlüğü.

06 Güvenlik + kanıt

HMAC/SSRF/secret koruması; kritik testler, log/metrik, Compose ve CI.

BONUS / ÇEKİRDEK KABUL ÖLÇÜTLERİ GEÇİNCE

Manuel replay

FAILED teslimatı yetkili ve audit kayıtlı yeniden planlama.

Çoklu organizasyon

Multi-project/tenancy; sahiplik ve veri izolasyonu.

İşletim ve arayüz

Panel, gelişmiş dashboard, Redis rate limit veya cloud deployment.

Kapsam kapısı

Otomatik retry ve çökme sonrası recovery zorunlu. Replay, panel ve cloud demo teslim şartı değil. Çekirdek doğrulandıktan sonra en fazla 1-2 bonus seç.

Bu takvimde hedefleme: Kafka, Kubernetes, çok bölge ve çok sayıda mikroservis. Bunlar çekirdeği tamamlamak için gerekli değildir.

Her aracın bir görevi olsun

Aynı teknoloji yığını; öğrenme konuları ve projedeki somut karşılıkları kartlarda.

Java 21 + Maven

Öğren: OOP, koleksiyonlar, exception ve HTTP.
Uygula: Event kuralları, durum modeli ve Maven Wrapper.

Spring Boot + Security

Öğren: REST, DI, DTO, Validation ve API erişimi.
Uygula: Endpoint/subscription API'si, ortak hata modeli, OpenAPI.

PostgreSQL + JPA + Flyway

Öğren: İlişkiler, SQL, transaction ve migration.
Uygula: FK/unique, atomik outbox ve sorgu index'leri.

Spring AMQP + RabbitMQ

Öğren: Producer/consumer, confirm ve ACK.
Uygula: Relay, delivery kuyruğu ve worker.

JUnit + Mockito

Öğren: Birim testi ve bağımlılıkları ayırma.
Uygula: Matching, retry ve durum kurallarını sınama.

Testcontainers + HTTP stub

Öğren: Gerçek DB/broker, kontrollü HTTP alıcısı.
Uygula: Rollback, duplicate, timeout ve 500 → 200 testleri.

Actuator + Micrometer

Öğren: Sağlık, yapılandırılmış log ve metrik.
Uygula: Teslim sonucu, retry sayısı ve outbox yaşı.

Docker Compose + CI

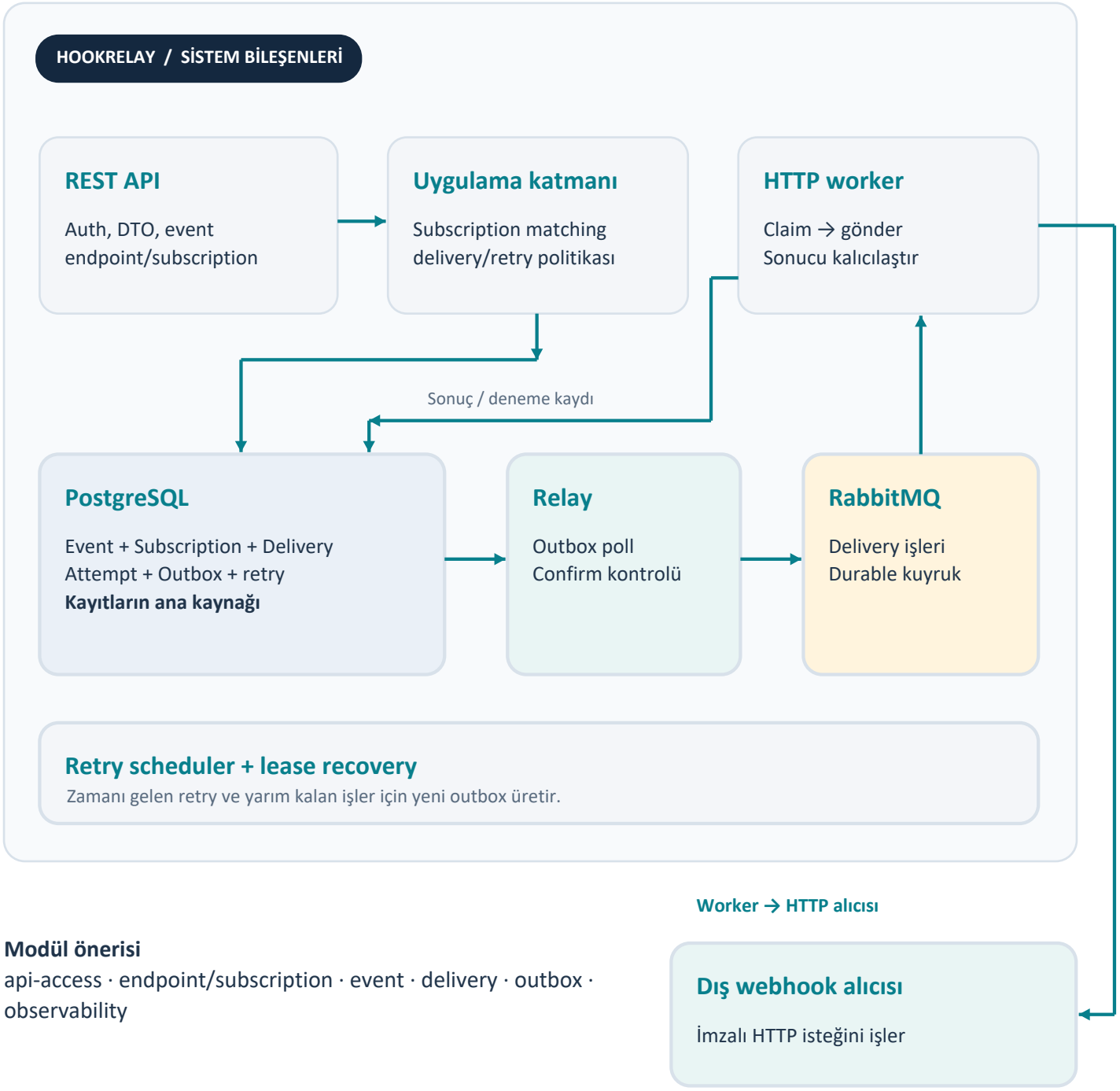
Öğren: Tekrarlanabilir kurulum ve otomatik test.
Uygula: Yerel demo; GitHub Actions build/test akışı.

Sürüm yaklaşımı

Java 21 LTS başlangıç hedefi. Spring Boot ve bağımlılıklarını uyumlu, kararlı sürümlerle sabitle; BOM kullan. Kesin sürümleri README'de yaz. Redis çekirdek bağımlılığı değildir. [1, 2]

Tek uygulama, açık sınırlar

Tek repo, modüler monolit ve tek organizasyon. API, relay ve worker ayrı sorumluluklar.



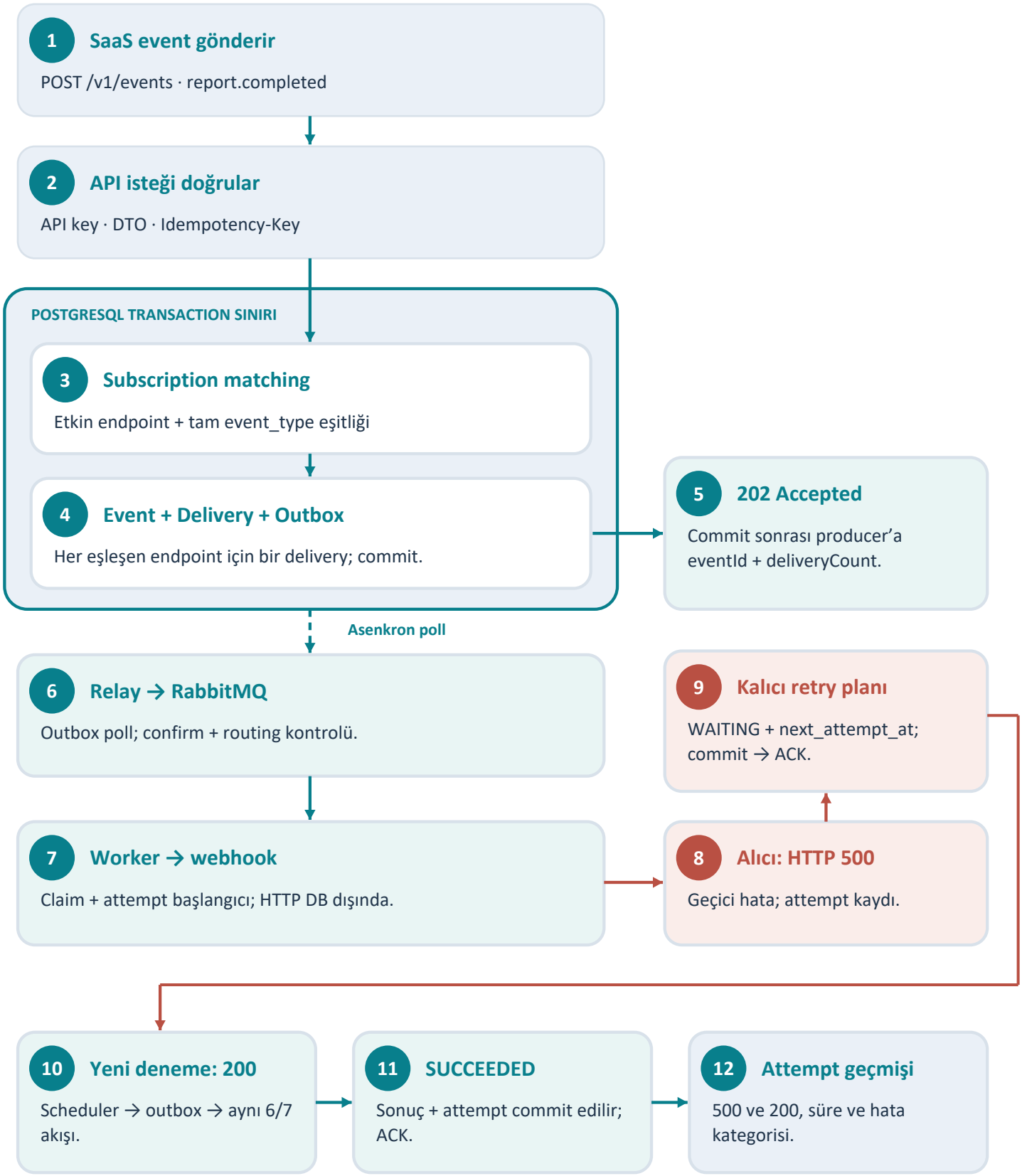
Modül önerisi
api-access · endpoint/subscription · event · delivery · outbox · observability

Sınırlar mimarinin parçasıdır

HTTP çağrısı boyunca DB transaction'ı açık tutma. PostgreSQL işlemi ile RabbitMQ publish'i aynı yerel transaction değildir; aradaki boşluğu outbox kapatır. [3, 4]

Bir webhook’un yolculuğu

report.completed: ilk gönderim 500, sonraki gönderim 200. Aynı deliveryId korunur.



Turkuaz: başarı · Mercan: retry. Relay, 202 yanıtından bağımsız olarak commit edilmiş outbox’ı tarar.

Transactional outbox nasıl işler?

“DB’ye yazdım ama kuyruğa gönderemedim” boşluğunu görünür ve toparlanabilir kıl.

TEK POSTGRESQL TRANSACTION’I

Event	Delivery	Outbox
İstek + payload	Hedef başına iş	Yayımlanacak iş

Kabul yanıtının anlamı

Transaction commit olunca 202 Accepted döndür: “Kalıcı olarak kabul edildi.” Bu yanıt, webhook’un alıcıya ulaştığını söylemez.

1. **Relay outbox işini claim eder**

   Kısa DB işlemi ve süreli lease. Birden fazla relay için SKIP LOCKED veya eşdeğer atomik claim.

2. **İşi RabbitMQ’ya yayımlar**

   Mesajda deliveryId + dispatchVersion. Durable kuyruk, persistent mesaj, publisher confirm ve routing kontrolü.

3. **Yayım sonucunu kaydeder**

   Pozitif confirm ve return olmaması sonrası published işaretlenir. Belirsiz sonuçta iş yeniden yayımlanabilir.

4. **Worker işler, sonra ACK verir**

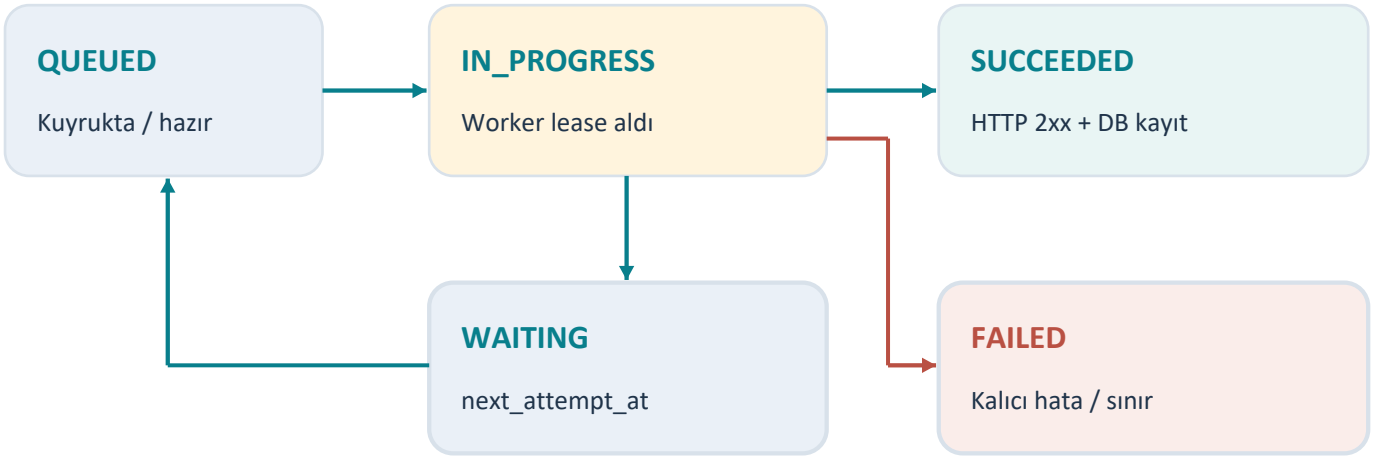
   Sonuç veya kalıcı retry kararı commit edilince ACK. Confirm broker kabulüdür; ACK, consumer’ın broker’a onayıdır.

Arıza pencerelerini bil

Publish sonrası, published kaydından önce çökme → aynı iş tekrar yayımlanabilir. HTTP 200 sonrası, sonuç kaydından önce çökme → aynı webhook tekrar gönderilebilir. [3, 4, 5]

Retry, idempotency ve recovery

Dağıtık bir teslimat sürecinde tekrar olabileceğini tasarımın başında kabul et.



Sonuç	Önerilen politika
2xx	Başarılı teslimat. Sonucu ve denemeyi kaydet.
Timeout, bağlantı, 408/429/5xx	Sınırlı retry. 429 için geçerli Retry-After değerini sınırla ve uygula.
Diğer 4xx / 3xx	Otomatik retry/redirect yok; FAILED DB durumudur. DLQ, bozuk/işlenemeyen kuyruk mesajlarını tutar.

Örnek retry: ilk deneme + en fazla 4 tekrar. Beklemeler 10 / 30 / 90 / 270 sn; küçük rastgele jitter ve üst sınır ekle. Zamanı DB'de tut; worker'ı bekleterek meşgul etme.

Kabul idempotency'si

Tek organizasyonda key benzersiz. Aynı key/içerik → aynı event; farklı içerik → 409.

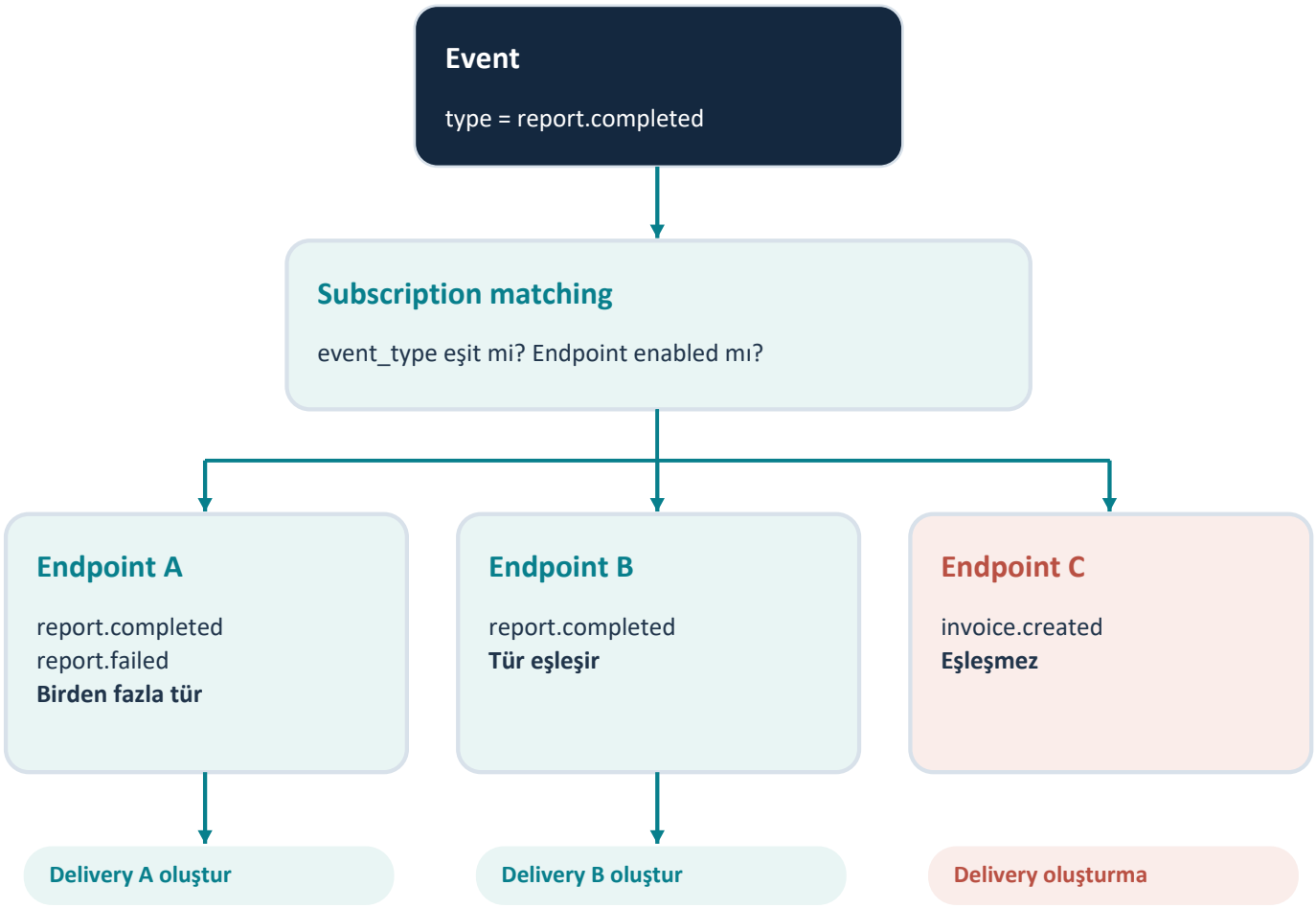
Teslimat idempotency'si

deliveryId sabit; stale dispatch'i ele. Alıcı da iş sonucunu idempotent kaydetmeli.

Recovery: süresi dolan IN_PROGRESS işi yeni dispatch sürümü + outbox ile atomik yeniden planlanır. Recovery de deneme sınırına uyar. Lease, HTTP deadline'dan uzun olmalı; eski worker sonucu claim token/sürüm ile reddedilir. **FAILED çekirdekte terminaldir; manuel replay bonustur.**

Yalnız ilgili hedeflere gönder

Önerilen karar: tam event_type eşleşmesi. Wildcard, regex ve ayrı abonelik servisi yok.



Basit ve ilişkisel model

EndpointSubscription: endpoint_id + event_type. Endpoint FK'si ve UNIQUE(endpoint_id, event_type) aynı aboneliğin iki kez kaydını engeller. Event türü doğrulanmış, büyük/küçük harfe duyarlı bir metindir. [5]

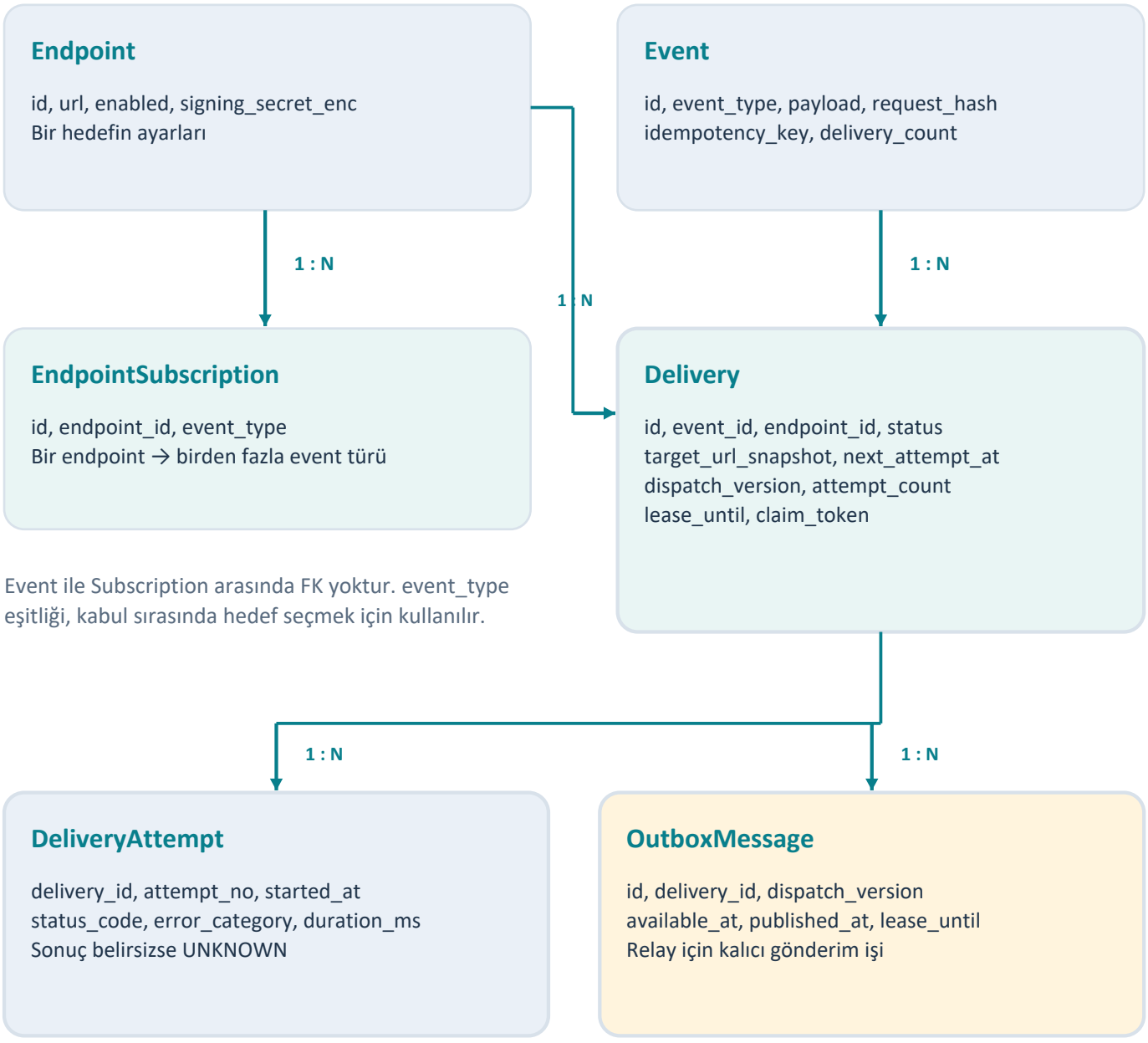
Hedef listesi kabulde sabitlenir

Kabul transaction'ındaki matching sorgusu hedefleri seçer. Subscription veya enabled değişikliği yalnız yeni event'leri etkiler; mevcut delivery'ler sürer. URL, delivery içinde snapshot olarak tutulur.

Eşleşme yok: Event yine kaydedilir; delivery/outbox oluşmaz, 202 yanıtında deliveryCount=0 görünür. Sonradan abonelik eklemek eski event'i dağıtmaz. Aynı Idempotency-Key, ilk event'i ve ilk hedef sayısını döndürür; yeniden matching yapmaz.

Subscription bir routing kuralıdır

Altı çekirdek tablo. Tek organizasyonda Project/tenant tablosu ilk sürüm için gerekli değil.



Event ile Subscription arasında FK yoktur. event_type eşitliği, kabul sırasında hedef seçmek için kullanılır.

Constraint ve index özeti

Unique: Event.idempotency_key; Subscription(endpoint_id, event_type); Delivery(event_id, endpoint_id); Outbox(delivery_id, dispatch_version); Attempt(delivery_id, attempt_no). FK/CHECK uygula. [5]

Index: subscription(event_type, endpoint_id); delivery(status, next_attempt_at); yayımlanmamış outbox(available_at). Zamanları UTC tut. API key doğrulaması hashli konfigürasyondan yapılabilir; secret rotation ve tenant modeli bonus kararlardır.

Sözleşmeyi açık yaz

Önerilen çekirdek API. Authentication her korumalı endpoint'te uygulanır.

Endpoint	Temel davranış
POST /v1/endpoints	HTTPS hedef oluştur; enabled ve URL kontrolü.
PATCH /v1/endpoints/{id}	URL/enabled değiştir; mevcut delivery hedefi sabit.
PUT /v1/endpoints/{id}/subscriptions	eventTypes listesi tüm abonelikleri atomik değiştirir.
POST /v1/events	Key + type + payload. 202: eventId, deliveryCount. Key çatışması 409.
GET /v1/deliveries	Status/eventId/tarih + sayfalama.
GET /v1/deliveries/{id}/attempts	Her denemenin sonucu, süresi ve hata kategorisi.

Subscription örneği: eventTypes = [report.completed, report.failed]

Gelen API isteği

Yüksek entropili API key'i hash olarak doğrula. Tek organizasyon bağlamı; public signup yok. DTO/payload limitleri ve 400/401/403/404 yanıtları tanımlı olsun.

Giden HMAC imzası

Timestamp + eventId + deliveryId + ham body, sabit ayraçlarla imzalanır. Alıcı sabit zamanlı karşılaştırma, zaman penceresi ve kalıcı dedupe uygular. HMAC şifreleme değildir.

SSRF: hedefi sınırla

Yönetilen HTTPS allowlist; IPv4/IPv6 özel ağ, loopback ve metadata çıkışını engelle. DNS ve gerçek bağlantı hedefini doğrula; redirect kapalı. HTTP süre/yanıt boyutu limitleri koy. Local mock istisnası yalnız dev profilinde. [6]

Secret ve bonus sınırı

HMAC secret'ı yeniden kullanılacağından şifreli sakla; çözme anahtarını repo dışında tut. Secret/payload loglama. **Manuel replay API'si bonus:** yetkili, audit kayıtlı; attempt geçmişi sıfırlanmaz.

Mutlu yol tek başına yetmez

Testleri ilk aydan yaz. Altıncı ayda gerçek altyapı ve hata senaryolarını derinleştir.

Birim: kurallar

Subscription matching; retry kararı; durum geçişleri ve hash karşılaştırma.
Araç: JUnit + Mockito

DB: bütünlük

Rollback yarım Event/Delivery/Outbox bırakmaz; eşzamanlı aynı key tek event üretir.
Araç: PostgreSQL + Testcontainers

Broker: kesintiler

Confirm belirsizliği, duplicate, stale dispatch, ACK öncesi çökme ve lease recovery.
Araç: RabbitMQ + Testcontainers

HTTP: alıcı davranışı

500 → 500 → 200; timeout/429; HMAC, SSRF, aynı deliveryId ve deneme sınırı.
Araç: HTTP stub + Compose

GÖRÜNÜRLÜK / ÜÇ KÜÇÜK GÖSTERGE

Teslimat

Başarı / hata sayısı
Hata kategorisi

Birikim

Kuyruk derinliği
En eski outbox yaşı

Zaman

Retry sayısı
Teslim gecikmesi

Log, metrik ve sağlık aynı şey değildir

Log: eventId, deliveryId, attempt, correlationId. Metrik etiketlerinde eventId kullanma; kardinalite büyür. Actuator'ı koru. DB/broker sağlığı ile teslim başarısını ayrı izle. Gelişmiş dashboard bonus. [7, 8]

Son kabul: temiz kurulum + CI + worker kapalı/açık demo + no-match routing + FAILED sınırı. Küçük yük testinde makine, payload, worker sayısı ve p50/p95 değerlerini yaz. Ölçmediğin throughput'u CV'ye koyma.

Mesajın yolculuğunu anla

Tanım → HookRelay'deki rol → küçük örnek. Consumer ile dış HTTP alıcısını ayır.

Event · Producer · Consumer

Event gerçekleşen olayın verisi; producer üretir, consumer kuyruktan okur.
HookRelay: SaaS producer; RabbitMQ worker'ı consumer'dır.
Örnek: report.completed → delivery işi → worker.

Webhook

Olay bilgisinin bir HTTP adresine gönderilmesidir.
HookRelay: Worker dış endpoint'e imzalı POST yapar; dış sistem webhook alıcısıdır.
Örnek: POST /hooks/reports → 200.

Message Broker · Queue

Broker mesajları yönetir; queue tüketilmeyi bekleyen mesajları tutar.
HookRelay: RabbitMQ iş bildirimini taşır; gerçek teslimat durumu PostgreSQL'dedir.
Örnek: Queue: deliveryId=42, dispatchVersion=2.

ACK · Publisher Confirm

Confirm broker'dan yayıncıya; ACK consumer'dan broker'a gider.
HookRelay: Relay confirm bekler; worker kalıcı sonuç/karar sonrası ACK verir.
Örnek: Confirm ≠ consumer ACK ≠ HTTP 200. [4]

Transactional Outbox

İş verisi ile yayımlanacak iş aynı DB transaction'ında saklanır.
HookRelay: Event + Delivery + Outbox commit; relay daha sonra publish eder.
Örnek: Rollback: hiçbiri kalmaz. Publish tekrar olabilir. [3]

At-least-once Delivery

Yeniden deneme/işleme sırasında aynı teslimat birden fazla kez gelebilir.
HookRelay: Kabul edilmiş iş kalıcı izlenir; sınırlı retry sonunda FAILED da olabilir.
Örnek: Alıcı işledi, yanıt kayboldu → aynı deliveryId tekrar. [4]

Hatırlatıcı: Producer → API/DB → relay → broker/queue → consumer(worker) → webhook alıcısı. Kuyruk mesajının işlenmesi ile uzak sistemdeki iş etkisi farklı sınırlardır.

Tekrar, zaman ve güvenlik

Bu kartlar retry/recovery sayfalarını okurken hızlı hatırlatma içindir.

Idempotency · Deduplication

İdempotency tekrarın ek etki üretmemesi; dedup kopyayı tanımaktır.
HookRelay: API key değil, Idempotency-Key isteği tekilleştirir.
Örnek: Aynı key/içerik → aynı event.

Retry · Backoff · Jitter

Retry tekrar dener; exponential backoff beklemeyi katlar; jitter rastgele farktır.
HookRelay: Bekleme zamanı DB'dedir; deneme sayısı sınırlı.
Örnek: 10 → 30 → 90 sn + küçük jitter.

Claim · Lease

Claim işi sahiplenir; lease sahipliği süreyle sınırlar.
HookRelay: DB'de atomik claim; HTTP DB transaction'ı dışında.
Örnek: Worker A, işi süreli alır.

Lease Recovery

Süresi dolmuş sahipliği fark edip yarım işi yeniden planlamaktır.
HookRelay: Yeni sürüm + outbox atomik; eski sonuç token ile reddedilir.
Örnek: Worker çöktü → lease doldu → yeni dispatch.

Stale Dispatch

Teslimatın güncel sürümünden eski kuyruk komutudur.
HookRelay: Version karşılaştır; tekrar HTTP başlatma.
Örnek: DB v3 iken kuyruktaki v2 elenir.

Dead Letter Queue (DLQ)

İşlenemeyen broker mesajlarının ayrıldığı kuyruk.
HookRelay: Bozuk mesajı ayırır; DB FAILED durumu farklıdır.
Örnek: Bozuk JSON → DLQ; HTTP 500 → retry. [9]

HMAC

Paylaşılan sırla içerik bütünlüğünü ve göndericiyi doğrulayan kod.
HookRelay: Webhook imzalanır; secret şifreli saklanır.
Örnek: İçerik değişirse imza tutmaz. HMAC şifreleme değildir.

SSRF

Sunucunun istenmeyen hedefe istek göndermesidir.
HookRelay: Hedef, DNS/bağlantı ve ağ çıkışı sınırlandırılır.
Örnek: Production'da localhost/metadata engellenir. [6]

Önce sağlam temeli kur

Haftada 8-10 saat varsayımı. Süreler kayabilir; ay sonu kabul ölçütlerini koru.

AY

1 2 3 4 5 6 7

1 AY 1 / Java temeli

Öğren Java, OOP, koleksiyonlar, exception, Maven, Git ve HTTP.

Uygula Küçük event doğrulama uygulaması; durum modeli, hata sınıfları ve ilk birim testleri.

Kanıtla Çalışan küçük uygulama + README'de net kapsam.

2 AY 2 / Spring REST API

Öğren Spring REST, DI, controller/service, DTO, validation, hata yanıtları.

Uygula In-memory endpoint/subscription/event API prototipi; OpenAPI ve basit API key kontrolü.

Kanıtla Matching ve REST demosu; controller testleri. Restart bellekteki veriyi siler.

3 AY 3 / Veri ve transaction

Öğren PostgreSQL, JPA, SQL, ilişkiler, unique/FK, index ve Flyway.

Uygula In-memory yerine JPA; Subscription ilişkileri; Event + Delivery + Outbox atomik kayıt.

Kanıtla Restart veriyi korur; rollback ve subscription/unique testleri geçer.

Haftalık ritim

İki akşam 2'şer saat + hafta sonu 4-6 saat. Bir oturum öğren, iki oturum uygula, son oturumda test et ve kısa not al. Her ay sonunda çalışan bir milestone göster.

Güvenilirliği görünür kıl

Yaklaşık 28 haftalık plan: 224-280 saat. Yedinci ayın en az iki haftası tampon.

AY



4 AY 4 / Asenkron teslimat

Öğren RabbitMQ, producer/consumer, confirm, ACK ve outbox.

Uygula Relay, worker, atomik claim/lease, temel recovery ve HTTP gönderimi.

Kanıtla Broker/worker kesintisi sonrası kabul edilen iş devam eder.

5 AY 5 / Retry ve güvenlik

Öğren Backoff/jitter, idempotency, API key, HMAC ve SSRF.

Uygula Kalıcı retry, FAILED, stale dispatch; recovery ve güvenlik testlerini tamamla.

Kanıtla 500 → 200 akışı, aynı key'in tek event üretmesi ve imza testi.

6 AY 6 / Test ve işletim

Öğren Testcontainers, yapılandırılmış log, metrik, CI ve küçük yük testi.

Uygula DB + broker + stub senaryoları; Compose ve otomatik build/test.

Kanıtla Temiz kurulum geçer; crash/duplicate kanıtları CI'da çalışır.

7 AY 7 / Teslim ve portföy

Öğren Refactoring, kararları anlatma ve sürümleme.

Uygula Hata kapatma, README, mimari diyagram, demo ve release.

Kanıtla Bir başkası README ile çalıştırır; v1.0.0 ve demo hazır.

Tempo düşerse sıralamayı koru

7. ayın en az iki haftasını test düzeltme, dokümantasyon, demo ve gecikmelere ayır. Replay/multi-tenancy/panel bonusları bu tamponu tüketmemeli.

Repo, sistemin hikâyesini anlatsın

Bir okuyucu birkaç dakikada problemi, mimari kararı ve çalışan kanıtı görebilmeli.

README'nin ilk ekranı

1. Problem ve tek cümlelik değer önerisi
2. Mimari diyagram + çekirdek özellikler
3. Compose + .env.example ile kurulum
4. Örnek API isteği ve demo bağlantısı
5. Test, sınırlar ve sürüm bilgisi

Dokümantasyon paketi

architecture.md - modüller ve veri modeli
reliability.md - outbox, retry, duplicate
security.md - key, HMAC, URL sınırı
decisions/ - kısa karar kayıtları
runbook.md - hata/recovery adımları

Commit ve sürüm geçmişi

Tek amacı olan doğal commitler kullan: "Persist event and outbox atomically", "Retry temporary delivery failures". Issue'lara kabul ölçütü yaz; ay sonunda milestone/release oluştur. Yapay commit sayısı üretme.

5-7 DAKİKALIK DEMO AKIŞI

01 Endpoint/subscription oluştur; report.completed gönder, eşleşen delivery'leri göster.

02 Alıcı 500 → 500 → 200 dönsün; retry aralıkları ve attempt geçmişini aç.

03 Aynı Idempotency-Key ile tekrar gönder; yeni event oluşmadığını göster.

04 Worker'ı durdur, event kabul et, yeniden aç; işlemenin devamını göster.

05 Aboneliği olmayan geçerli event'te deliveryCount=0; sürekli hatada retry → FAILED göster.

CV'de kanıtlayabildiğin davranışı yaz

Örnek ifade: "Spring Boot, PostgreSQL ve RabbitMQ ile webhook teslimat platformu geliştirdim; outbox, kalıcı retry ve HMAC imzalı gönderimleri hata senaryolarıyla test ettim." Yalnız tamamladığın ve gösterebildiğin kısmı kullan.

Bittiğini nasıl anlayacaksın?

Çekirdek kabul listesi. Her işaret bir çalışan davranış ve test kanıtına dayanmalı.

- [ ] Temiz kurulum, migration, Compose ve CI testleri çalışıyor.
- [ ] Subscription routing doğru; ilgisiz event hedefe gitmiyor.
- [ ] Event/Delivery/Outbox atomik; aynı key yeniden matching yapmıyor.
- [ ] Broker/worker kesintisi, stale dispatch ve lease recovery testli.
- [ ] Retry sınırlı/kalıcı; FAILED ve attempt geçmişi sorgulanabiliyor.
- [ ] API key, HMAC, SSRF, secret koruması ve portföy demosu hazır.

Temel güvenilirlik sözleşmesi

At-least-once: tekrar teslim mümkündür. Confirm, consumer ACK ve HTTP 200 farklı onaylardır. HTTP 200 sonrası sonuç commit'inden önce worker çökerse aynı webhook yeniden gönderilebilir. Exactly-once iddiası yok. [4]

KAYNAKLAR / TIKLANABİLİR RESMÎ BELGELER

[1] Java 21 API belgeleri

[2] Spring Boot - Java ve sürüm uyumluluğu

[3] AWS - Transactional outbox tasarım örüntüsü

[4] RabbitMQ - Consumer ACK ve publisher confirm

[5] PostgreSQL - Constraints, FK ve unique

[6] OWASP - SSRF önleme rehberi

[7] Testcontainers - PostgreSQL entegrasyon modülü

[8] Spring Boot - Actuator metrikleri

[9] RabbitMQ - Dead letter exchanges / DLQ

Ek: RabbitMQ Reliability · PostgreSQL SKIP LOCKED

StockFlow kaynakları: product-spec.md (3 Eylül 2026) ve StockFlow Proje Tanıtım ve Özellikler v1.1 (18 Ağustos 2026). Karşılaştırma kapsam belgelerine dayanır; mevcut uygulamanın tamamlandığı varsayılmaz.

Bu rapor bir geliştirme rehberidir. Subscription kuralları, tek organizasyon sınırı, API/şema ayrıntıları ve retry değerleri HookRelay için önerilen tasarım kararlarıdır.